PES UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Laboratory

Practical Git & GitHub Workflow Demonstration Report

Student Name:	Abhijna Marathe
Student SRN:	PES2UG24CS020
Course Title:	Software Engineering (SE)
Course Code:	UE24CS351A
GitHub Profile:	https://github.com/ack-chai
GitHub Repository:	https://github.com/ack-chai/demo
Submission Date:	October 20, 2026

Executive Summary: *This lab report documents the comprehensive execution of all core Git and GitHub workflows stipulated in the Git Demo cheatsheet. Operations span local repository creation, staging, atomic commits, uncommitted change inspection, file restoration, divergent branch creation, manual merge conflict resolution, remote synchronization with GitHub (push, fetch, pull, clone), and branch deletion. The live repository is publicly hosted at* ***https://github.com/ack-chai/demo****.*

1. Configuration & Local Repository Creation

A local project directory demo was created and initialized as a Git repository. An initial source file app.py was created, staged, and committed as the base snapshot.

Executed Commands & Output:

```
$ mkdir demo && cd demo
$ git init
Initialized empty Git repository in /Users/abhijna/.gemini/antigravity-ide/scratch/demo/.git/
$ echo 'print("Initial demo")' > app.py
$ git status
On branch main
Untracked files: app.py
$ git add app.py && git add .
$ git commit -m "Initial commit"
[main (root-commit) 783bd86] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 app.py
$ git log --oneline
783bd86 (HEAD -> main) Initial commit
```

2. Inspecting and Undoing Changes

To demonstrate working directory inspection and state recovery without loss of data, modifications were introduced to app.py and inspected before staging, after staging, and subsequently restored.

Executed Commands & Output:

```
# Modify app.py with uncommitted edits
$ git diff
diff --git a/app.py b/app.py
--- a/app.py
+++ b/app.py
@@ -1 +1,2 @@
print("Initial demo")
+print("Testing uncommitted change")
# Stage changes and inspect staged diff
$ git add app.py
$ git diff --staged
# Unstage changes without losing edits
$ git restore --staged app.py
# Discard uncommitted changes, reverting to the last commit
$ git restore app.py
```

3. Branching, Merging, and Conflict Resolution

A feature branch feature was created to develop changes independently. Meanwhile, divergent modifications were committed directly to main on the same line in app.py to intentionally provoke a merge conflict. The conflict markers were inspected, manually resolved, staged, and committed to complete a two-parent merge.

Conflict Trigger and Manual Resolution:

```
$ git checkout -b feature
$ echo 'print("Feature update")' > app.py
$ git commit -am "Feature commit"
$ git checkout main
$ echo 'print("Main update")' > app.py
$ git commit -am "Main commit"
$ git merge feature
Auto-merging app.py
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
```

Conflict Marker Resolution in app.py:

```
# Inside app.py during conflict:
<<<<<<< HEAD
print("Main update")
=======
print("Feature update")
>>>>>>> feature
# Resolved manually to:
print("Resolved feature in main")
$ git add app.py
$ git commit -m "Merge branch 'feature'"
[main ef342e3] Merge branch 'feature'
$ git branch -d feature
Deleted branch feature (was c94b854).
```

4. Remote Operations & GitHub Synchronization

The local repository was renamed to main, connected to GitHub, and pushed to the remote repository **https://github.com/ack-chai/demo**. Synchronization operations (fetch, pull, clone, and remote branch deletion) were verified.

Executed Commands & Output:

```
$ git branch -M main
$ gh repo create demo --public --source=. --remote=origin --push
✓ Created repository ack-chai/demo on github.com
✓ Added remote https://github.com/ack-chai/demo.git
✓ Pushed commits to https://github.com/ack-chai/demo.git
$ git remote -v
origin https://github.com/ack-chai/demo.git (fetch)
origin https://github.com/ack-chai/demo.git (push)
$ git fetch origin
$ git pull origin main
Already up to date.
# Verification of remote branch push and deletion:
$ git checkout -b demo-feature && git push origin demo-feature
$ git checkout main && git push origin --delete demo-feature
To https://github.com/ack-chai/demo.git
- [deleted] demo-feature
```

5. Commit Graph and Final Repository State

The verified commit graph confirms the complete merge history and subsequent documentation additions:

```
$ git log --graph --oneline --all
* cd84f5f (HEAD -> main, origin/main) Add comprehensive README documentation
* ef342e3 Merge branch 'feature'
|\
| * c94b854 Feature commit
```

```
* | a0c92b3 Main commit
|/
* 783bd86 Initial commit
```

6. Conceptual Foundations (Cheatsheet Page 3)

6.1 Version Control

A **Version Control System (VCS)** is a software system that systematically records every change made to files and directories over time within a project repository. It provides complete provenance and auditability—enabling developers to see who modified what line of code, why the change occurred, and when it was committed. If defects arise, developers can instantly roll back files to any previous known good state. Crucially, a distributed VCS allows multiple engineers to work on identical files simultaneously on isolated branches without overwriting or corrupting one another's contributions.

6.2 Git vs. GitHub

Git is an open-source, distributed version control tool executed directly from the developer's computer. It runs completely offline, maintaining a local database in the hidden .git folder with the full snapshot history of the project.
GitHub is a cloud-based hosting service and collaboration platform built around Git. It stores remote replicas of Git repositories and enriches them with collaboration tooling that Git alone does not possess: web-based code reviews, Pull Requests, issue trackers, discussions, project boards, and CI/CD pipelines (GitHub Actions). In essence, Git is the version control engine; GitHub is the platform that hosts Git repositories in the cloud.

6.3 Fork

A **Fork** is a complete, independent copy of another user's remote Git repository created under your own personal GitHub account. Forks are standard in open-source workflows when an external developer does not possess write/push permissions to the original repository. By forking the project, the developer can freely clone their copy, create branches, push experimental commits, and thoroughly test features. When the contribution is polished and verified, the developer submits a **Pull Request (PR)** from their fork back to the original repository, inviting the project maintainers to review and merge the proposed changes.

7. Conclusion

All commands and conceptual requirements specified in the Software Engineering Git Demo cheatsheet have been successfully completed, empirically validated, and pushed to GitHub. The live repository is publicly available for evaluation at: **https://github.com/ack-chai/demo**.